Hot-Swappable Configuration System

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

int *active_config = NULL;
pthread_mutex_t config_mutex = PTHREAD_MUTEX_INITIALIZER;

void* reader(void* arg) {
    int id = *((int*)arg);
    while (1) {
        pthread_mutex_lock(&config_mutex);

        int value = *active_config;
pthread_mutex_unlock(&config_mutex);

        printf("Reader %d read config value: %d\n", id, value);

        sleep(1);
    }
    return NULL;
}

void* updater(void* arg) {
    while (1) {
        sleep(3);  // Simulate periodic update
        pthread_mutex_lock(&config_mutex);

        // Allocate new config first
        int *new_config = malloc(sizeof(int));
        *new_config = rand() % 100;

        // Save old config
        int *old_config = active_config;

        // Atomic swap
        active_config = new_config;

        free(old_config);

        pthread_mutex_unlock(&config_mutex);

        printf("New config: %d\n", *new_config);
```

```
    }
    return NULL;
}

int main() {
    srand(time(NULL));

    active_config = malloc(sizeof(int));
    *active_config = 10;

    pthread_t r1, r2, r3, up;
    int id1 = 1, id2 = 2, id3 = 3;

    pthread_create(&r1, NULL, reader, &id1);
    pthread_create(&r2, NULL, reader, &id2);
    pthread_create(&r3, NULL, reader, &id3);
    pthread_create(&up, NULL, updater, NULL);

    pthread_join(r1, NULL);
    pthread_join(r2, NULL);
    pthread_join(r3, NULL);
    pthread_join(up, NULL);
    return 0;
}
```

Dining Philosophers (Limited Seating)

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

pthread_mutex_t forks[N];   // One mutex per fork
sem_t room;                 // Counting semaphore (max 4 philosophers)

void* philosopher(void* arg) {

    int id = *((int*)arg);
    int left = id;
    int right = (id + 1) % N;

    while (1) {
```

```
        printf("Philosopher %d is thinking\n", id);
        sleep(1);

        // Enter room (limit 4)
        sem_wait(&room);

        // Pick up forks
        pthread_mutex_lock(&forks[left]);
        pthread_mutex_lock(&forks[right]);

        printf("Philosopher %d is eating\n", id);
        sleep(2);

        // Put down forks
        pthread_mutex_unlock(&forks[right]);
        pthread_mutex_unlock(&forks[left]);

        // Leave room
        sem_post(&room);
    }
    return NULL;
}

int main() {

    pthread_t phil[N];
    int ids[N];

    // Initialize room semaphore to 4
    sem_init(&room, 0, 4);

    // Initialize fork mutexes
    for (int i = 0; i < N; i++)
        pthread_mutex_init(&forks[i], NULL);

    for (int i = 0; i < N; i++) {
        ids[i] = i;
        pthread_create(&phil[i], NULL, philosopher, &ids[i]);
    }

    for (int i = 0; i < N; i++)
        pthread_join(phil[i], NULL);
    return 0;
}
```

Thread-Safe Histogram (Sharded Updates)

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define SIZE 9
#define BUCKETS 10
#define THREADS 2

int data[SIZE] = {1, 2, 3, 4, 2, 5, 6, 2, 1};
int histogram[BUCKETS] = {0};

pthread_mutex_t lock;

typedef struct {
    int start;
    int end;
} ThreadArgs;

void* compute_histogram(void* arg) {

    ThreadArgs* args = (ThreadArgs*)arg;
    int local_hist[BUCKETS] = {0};

    // Build local histogram
    for (int i = args->start; i <= args->end; i++) {
        int value = data[i];
        local_hist[value]++;
    }

    // Merge into global histogram (critical section)
    pthread_mutex_lock(&lock);

    for (int i = 0; i < BUCKETS; i++) {
        histogram[i] += local_hist[i];
    }

    pthread_mutex_unlock(&lock);

    return NULL;
}

int main() {
```

```
    pthread_t threads[THREADS];
    ThreadArgs args[THREADS];

    pthread_mutex_init(&lock, NULL);

    // Divide work
    args[0].start = 0;
    args[0].end = 4;

    args[1].start = 5;
    args[1].end = 8;

    // Create threads
    for (int i = 0; i < THREADS; i++) {
        pthread_create(&threads[i], NULL, compute_histogram, &args[i]);
    }

    // Join threads
    for (int i = 0; i < THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    // Print histogram
    int total = 0;
    printf("Final Histogram:\n");
    for (int i = 0; i < BUCKETS; i++) {
        printf("Value %d: %d\n", i, histogram[i]);
        total += histogram[i];
    }

    printf("Total count = %d\n", total);

    pthread_mutex_destroy(&lock);

    return 0;
}
```

H_2O Bond Formation

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
```

```
#include <semaphore.h>
#include <unistd.h>

sem_t h_sem, o_sem, barrier;

int h_count = 0;
int o_count = 0;

void* hydrogen(void* arg) {

    printf("H Ready\n");
    h_count++;

    if (h_count >= 2 && o_count >= 1) {
        sem_post(&h_sem);
        sem_post(&h_sem);
        sem_post(&o_sem);

        h_count -= 2;
        o_count -= 1;
    }

    sem_wait(&h_sem);

    printf("H Bonded\n");

    sem_post(&barrier);
    sem_wait(&barrier);
    return NULL;
}

void* oxygen(void* arg) {

    printf("O Ready\n");
    o_count++;

    if (h_count >= 2) {
        sem_post(&h_sem);
        sem_post(&h_sem);
        sem_post(&o_sem);

        h_count -= 2;
        o_count -= 1;
    }
```

```
    sem_wait(&o_sem);

    printf("O Bonded\n");

    sem_post(&barrier);
    sem_wait(&barrier);
    return NULL;
}

int main() {

    pthread_t h[6], o[3];

    sem_init(&h_sem, 0, 0);
    sem_init(&o_sem, 0, 0);
    sem_init(&barrier, 0, 0);

    // Create hydrogen threads
    for (int i = 0; i < 6; i++)
        pthread_create(&h[i], NULL, hydrogen, NULL);

    sleep(1); // Optional ordering effect

    // Create oxygen threads
    for (int i = 0; i < 3; i++)
        pthread_create(&o[i], NULL, oxygen, NULL);

    for (int i = 0; i < 6; i++)
        pthread_join(h[i], NULL);

    for (int i = 0; i < 3; i++)
        pthread_join(o[i], NULL);

    return 0;
}
```

Multi-Account Transfer System

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

int A = 1000, B = 1000, C = 1000;
```

```
pthread_mutex_t mA = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mB = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mC = PTHREAD_MUTEX_INITIALIZER;

void lock_accounts(pthread_mutex_t *m1, pthread_mutex_t *m2) {
    if (m1 < m2) {
        pthread_mutex_lock(m1);
        pthread_mutex_lock(m2);
    } else {
        pthread_mutex_lock(m2);
        pthread_mutex_lock(m1);
    }
}

void unlock_accounts(pthread_mutex_t *m1, pthread_mutex_t *m2) {
    pthread_mutex_unlock(m1);
    pthread_mutex_unlock(m2);
}

void transfer(int *from, int *to, int amount,
              pthread_mutex_t *m_from,
              pthread_mutex_t *m_to,
              const char *name) {

    lock_accounts(m_from, m_to);
    if (*from >= amount) {
        printf("%s: Transferring %d\n", name, amount);
        *from -= amount;
        *to += amount;
    }
    unlock_accounts(m_from, m_to);
}

void* T1(void* arg) {
    for (int i = 0; i < 10; i++) {
        transfer(&A, &B, 100, &mA, &mB, "T1 A->B");
        usleep(100000);
    }
    printf("T1 Done | A=%d B=%d C=%d\n", A, B, C);
    return NULL;
}

void* T2(void* arg) {
    for (int i = 0; i < 20; i++) {
        transfer(&B, &C, 50, &mB, &mC, "T2 B->C");
```

```
        usleep(100000);
    }
    printf("T2 Done | A=%d B=%d C=%d\n", A, B, C);
    return NULL;
}

void* T3(void* arg) {
    for (int i = 0; i < 40; i++) {
        transfer(&C, &A, 25, &mC, &mA, "T3 C->A");
        usleep(100000);
    }
    printf("T3 Done | A=%d B=%d C=%d\n", A, B, C);
    return NULL;
}

int main() {
    pthread_t t1, t2, t3;
    pthread_create(&t1, NULL, T1, NULL);
    pthread_create(&t2, NULL, T2, NULL);
    pthread_create(&t3, NULL, T3, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);

    printf("Final State: A=%d B=%d C=%d\n", A, B, C);
    printf("Total = %d\n", A + B + C);
    return 0;
}
```

Limited-Capacity Transaction Server

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>
#include <stdbool.h>

#define CLIENTS 6
#define SERVER_CAPACITY 3

sem_t server_sem; //Counting sem
pthread_mutex_t mutex; // Protect shared data
```

```
bool serving[CLIENTS] = {false};  // Track client status
int finished = 0;                 // Track completed clients

void* client(void* arg) {
    int id = *(int*)arg;

    // Wait for available slot
    sem_wait(&server_sem);

    pthread_mutex_lock(&mutex);
    serving[id] = true;
    pthread_mutex_unlock(&mutex);

    sleep(1);  // Simulate transaction processing

    pthread_mutex_lock(&mutex);
    serving[id] = false;
    finished++;
    pthread_mutex_unlock(&mutex);

    sem_post(&server_sem);  // Release slot
    return NULL;
}

void* monitor(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);

        // Print status line
        for (int i = 0; i < CLIENTS; i++) {
            printf("C%d=%s", i + 1,
                   serving[i] ? "True" : "False");
            if (i != CLIENTS - 1)
                printf(", ");
        }
        printf("\n");

        // Stop monitoring when all clients finish
        if (finished == CLIENTS) {
            pthread_mutex_unlock(&mutex);
            break;
        }

        pthread_mutex_unlock(&mutex);
```

```
        usleep(300000);  // 0.3 sec delay for readable output
    }
    return NULL;
}

int main() {
    pthread_t clients[CLIENTS];
    pthread_t monitor_thread;
    int ids[CLIENTS];

    sem_init(&server_sem, 0, SERVER_CAPACITY);
    pthread_mutex_init(&mutex, NULL);

    // Create monitor thread
    pthread_create(&monitor_thread, NULL, monitor, NULL);

    // Create client threads
    for (int i = 0; i < CLIENTS; i++) {
        ids[i] = i;
        pthread_create(&clients[i], NULL, client, &ids[i]);
        usleep(100000);  // Small arrival gap
    }

    // Join client threads
    for (int i = 0; i < CLIENTS; i++) {
        pthread_join(clients[i], NULL);
    }

    pthread_join(monitor_thread, NULL);

    sem_destroy(&server_sem);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

Sequential Launch of Space Systems

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

// Semaphores
```

```
sem_t sem_nav;
sem_t sem_launch;

void* fuel_system(void* arg) {
    sleep(1);  // Simulate delay

    printf("[Fuel Thread] Fuel system activated. Signal sent.\n");

    sem_post(&sem_nav);   // Signal Navigation

    return NULL;
}

void* navigation_system(void* arg) {

    printf("[Navigation Thread] Waiting for fuel system...\n");

    sem_wait(&sem_nav);   // Wait for Fuel

    sleep(1);  // Simulate processing

    printf("[Navigation Thread] Navigation system online. Signal sent.\n");

    sem_post(&sem_launch);  // Signal Launch

    return NULL;
}

void* launch_system(void* arg) {

    printf("[Launch Thread] Waiting for navigation system...\n");

    sem_wait(&sem_launch);  // Wait for Navigation

    sleep(1);  // Simulate processing

    printf("[Launch Thread] Launch sequence initiated!\n");

    return NULL;
}

int main() {

    pthread_t fuel, navigation, launch;
```

```
    // Initialize semaphores to 0 (blocked initially)
    sem_init(&sem_nav, 0, 0);
    sem_init(&sem_launch, 0, 0);

    // Start threads in RANDOM order
    pthread_create(&navigation, NULL, navigation_system, NULL);
    pthread_create(&launch, NULL, launch_system, NULL);
    pthread_create(&fuel, NULL, fuel_system, NULL);

    pthread_join(fuel, NULL);
    pthread_join(navigation, NULL);
    pthread_join(launch, NULL);

    sem_destroy(&sem_nav);
    sem_destroy(&sem_launch);

    return 0;
}
```

Readers-Writers Synchronization Problem

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define READERS 3
#define WRITERS 1

int data = 0;            // Shared resource
int read_count = 0;      // Number of active readers

sem_t mutex;             // Protects read_count
sem_t write_block;       // Controls access to shared data

// ---------------- READER ----------------
void* reader(void* arg) {
    int id = *(int*)arg;

    printf("[Reader %d] Waiting to read...\n", id);

    // Entry section
    sem_wait(&mutex);
    read_count++;
```

```
    if (read_count == 1) {
        // First reader blocks writers
        sem_wait(&write_block);
    }
    sem_post(&mutex);

    // Critical section (Reading)
    printf("[Reader %d] Reading data: %d\n", id, data);
    sleep(1);

    // Exit section
    sem_wait(&mutex);
    read_count--;

    if (read_count == 0) {
        // Last reader allows writers
        sem_post(&write_block);
    }
    sem_post(&mutex);

    return NULL;
}

// ---------------- WRITER ----------------
void* writer(void* arg) {
    int id = *(int*)arg;

    printf("[Writer %d] Waiting to write...\n", id);

    sem_wait(&write_block);   // Only one writer OR no readers

    // Critical section (Writing)
    printf("[Writer %d] Writing data...\n", id);
    sleep(1);
    data++;
    printf("[Writer %d] Updated data to %d\n", id, data);

    sem_post(&write_block);

    return NULL;
}

// ---------------- MAIN ----------------
int main() {
```

```
    pthread_t r[READERS], w[WRITERS];
    int r_ids[READERS], w_ids[WRITERS];

    sem_init(&mutex, 0, 1);
    sem_init(&write_block, 0, 1);

    // Create threads in random order
    for (int i = 0; i < READERS; i++) {
        r_ids[i] = i + 1;
        pthread_create(&r[i], NULL, reader, &r_ids[i]);
        usleep(200000);
    }

    for (int i = 0; i < WRITERS; i++) {
        w_ids[i] = i + 1;
        pthread_create(&w[i], NULL, writer, &w_ids[i]);
        usleep(200000);
    }

    for (int i = 0; i < READERS; i++)
        pthread_join(r[i], NULL);

    for (int i = 0; i < WRITERS; i++)
        pthread_join(w[i], NULL);

    sem_destroy(&mutex);
    sem_destroy(&write_block);

    return 0;
}
```

Priority Logging System

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <string.h>
#include <unistd.h>

pthread_mutex_t mutex;

// Buffers for logs
char high_logs[10][100];
char low_logs[10][100];
```

```
int high_count = 0;
int low_count = 0;

// ---------------- LOW PRIORITY THREAD ----------------
void* low_priority(void* arg) {
    char* message = (char*)arg;

    pthread_mutex_lock(&mutex);

    strcpy(high_logs[0], "");  // No effect, just to emphasize no mixup
    strcpy(low_logs[low_count], message);
    low_count++;

    pthread_mutex_unlock(&mutex);

    return NULL;
}

// ---------------- HIGH PRIORITY THREAD ----------------
void* high_priority(void* arg) {
    char* message = (char*)arg;

    pthread_mutex_lock(&mutex);

    strcpy(high_logs[high_count], message);
    high_count++;

    pthread_mutex_unlock(&mutex);

    return NULL;
}

// ---------------- MAIN ----------------
int main() {

    pthread_t t1, t2, t3, t4;

    pthread_mutex_init(&mutex, NULL);

    // Create threads (random order)
    pthread_create(&t1, NULL, low_priority, "[LOW] Network OK");
    pthread_create(&t2, NULL, high_priority, "[HIGH] Disk Failure");
    pthread_create(&t3, NULL, low_priority, "[LOW] User Login");
    pthread_create(&t4, NULL, high_priority, "[HIGH] Power Surge");
```

```
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);

    // Dump logs in priority order
    printf("\n=== Buffer Dump ===\n");

    for (int i = 0; i < high_count; i++)
        printf("%s\n", high_logs[i]);

    for (int i = 0; i < low_count; i++)
        printf("%s\n", low_logs[i]);

    pthread_mutex_destroy(&mutex);

    return 0;
}
```

Ordered Startup System

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

// Semaphores
sem_t sem_comm;
sem_t sem_control;

// ---------------- POWER THREAD ----------------
void* power_system(void* arg) {
    sleep(1);  // Simulate delay

    printf("[Power Thread] Power Initialized. Signal sent.\n");

    sem_post(&sem_comm);   // Allow Communication to proceed

    return NULL;
}

// ---------------- COMMUNICATION THREAD ----------------
void* communication_system(void* arg) {
```

```
    printf("[Comm Thread] Waiting for Signal...\n");

    sem_wait(&sem_comm);   // Wait for Power

    sleep(1);

    printf("[Comm Thread] Communication Initialized. Signal sent.\n");

    sem_post(&sem_control);   // Allow Control to proceed

    return NULL;
}

// ---------------- CONTROL THREAD ----------------
void* control_system(void* arg) {

    printf("[Control Thread] Waiting for Signal...\n");

    sem_wait(&sem_control);   // Wait for Communication

    sleep(1);

    printf("[Control Thread] Control System Initialized.\n");

    return NULL;
}

// ---------------- MAIN ----------------
int main() {

    pthread_t power, comm, control;

    // Initialize semaphores to 0 (blocked state)
    sem_init(&sem_comm, 0, 0);
    sem_init(&sem_control, 0, 0);

    // Start threads in random order
    pthread_create(&control, NULL, control_system, NULL);
    pthread_create(&comm, NULL, communication_system, NULL);
    pthread_create(&power, NULL, power_system, NULL);

    pthread_join(power, NULL);
    pthread_join(comm, NULL);
    pthread_join(control, NULL);
```

```
    sem_destroy(&sem_comm);
    sem_destroy(&sem_control);

    return 0;
}
```

FCFS

```
void fcfs(vector<Process>& processes) {

    sort(processes.begin(), processes.end(),
         [](Process a, Process b) {
             return a.at < b.at;
         });

    int currentTime = 0;
    double totalTAT = 0, totalWT = 0;

    for (auto &p : processes) {

        if (currentTime < p.at)
            currentTime = p.at;

        currentTime += p.bt;

        p.ct = currentTime;
        p.tat = p.ct - p.at;
        p.wt = p.tat - p.bt;

        totalTAT += p.tat;
        totalWT += p.wt;
    }

    cout << "\nID\tAT\tBT\tCT\tTAT\tWT\n";
    for (auto p : processes)
        cout << p.id << "\t" << p.at << "\t" << p.bt << "\t"
             << p.ct << "\t" << p.tat << "\t" << p.wt << "\n";

    cout << "\nAverage TAT = " << totalTAT/processes.size();
    cout << "\nAverage WT = " << totalWT/processes.size() << endl;
}
```

Non-preemptive SJF

```
void sjf(vector<Process>& processes) {
```

```
int n = processes.size();
int currentTime = 0, completed = 0;
vector<bool> done(n, false);

double totalTAT = 0, totalWT = 0;

while (completed < n) {

    int idx = -1;
    int minBT = 1e9;

    for (int i = 0; i < n; i++) {
        if (!done[i] && processes[i].at <= currentTime) {
            if (processes[i].bt < minBT) {
                minBT = processes[i].bt;
                idx = i;
            }
        }
    }

    if (idx == -1) {
        currentTime++;
        continue;
    }

    currentTime += processes[idx].bt;

    processes[idx].ct = currentTime;
    processes[idx].tat = processes[idx].ct - processes[idx].at;
    processes[idx].wt = processes[idx].tat - processes[idx].bt;

    totalTAT += processes[idx].tat;
    totalWT += processes[idx].wt;

    done[idx] = true;
    completed++;
}

cout << "\nID\tAT\tBT\tCT\tTAT\tWT\n";
for (auto p : processes)
    cout << p.id << "\t" << p.at << "\t" << p.bt << "\t"
         << p.ct << "\t" << p.tat << "\t" << p.wt << "\n";

cout << "\nAverage TAT = " << totalTAT/n;
```

```
    cout << "\nAverage WT = " << totalWT/n << endl;
}
```

Preemptive SJF

```
void preemptiveSJF(vector<Process>& processes) {

    int n = processes.size();
    int currentTime = 0, completed = 0;

    for (int i = 0; i < n; i++)
        processes[i].remaining = processes[i].bt;

    double totalTAT = 0, totalWT = 0;

    while (completed < n) {

        int idx = -1;
        int minRemaining = 1e9;

        for (int i = 0; i < n; i++) {
            if (processes[i].at <= currentTime &&
                processes[i].remaining > 0) {

                if (processes[i].remaining < minRemaining) {
                    minRemaining = processes[i].remaining;
                    idx = i;
                }
            }
        }

        if (idx == -1) {
            currentTime++;
            continue;
        }

        processes[idx].remaining--;
        currentTime++;

        if (processes[idx].remaining == 0) {

            processes[idx].ct = currentTime;
            processes[idx].tat = processes[idx].ct - processes[idx].at;
            processes[idx].wt = processes[idx].tat - processes[idx].bt;
```

```
            totalTAT += processes[idx].tat;
            totalWT += processes[idx].wt;
            completed++;
        }
    }

    cout << "\nID\tAT\tBT\tCT\tTAT\tWT\n";
    for (auto p : processes)
        cout << p.id << "\t" << p.at << "\t" << p.bt << "\t"
             << p.ct << "\t" << p.tat << "\t" << p.wt << "\n";

    cout << "\nAverage TAT = " << totalTAT/n;
    cout << "\nAverage WT = " << totalWT/n << endl;
}
```

Round Robin

```
void roundRobin(vector<Process>& processes, int quantum) {

    int n = processes.size();
    queue<int> q;
    int currentTime = 0, completed = 0;

    for (int i = 0; i < n; i++)
        processes[i].remaining = processes[i].bt;

    vector<bool> inQueue(n, false);

    double totalTAT = 0, totalWT = 0;

    while (completed < n) {

        for (int i = 0; i < n; i++) {
            if (processes[i].at <= currentTime &&
                !inQueue[i] && processes[i].remaining > 0) {

                q.push(i);
                inQueue[i] = true;
            }
        }

        if (q.empty()) {
            currentTime++;
            continue;
        }
```

```
        int idx = q.front();
        q.pop();

        int execTime = min(quantum, processes[idx].remaining);

        processes[idx].remaining -= execTime;
        currentTime += execTime;

        for (int i = 0; i < n; i++) {
            if (processes[i].at <= currentTime &&
                !inQueue[i] && processes[i].remaining > 0) {

                q.push(i);
                inQueue[i] = true;
            }
        }

        if (processes[idx].remaining > 0) {
            q.push(idx);
        } else {
            processes[idx].ct = currentTime;
            processes[idx].tat = processes[idx].ct - processes[idx].at;
            processes[idx].wt = processes[idx].tat - processes[idx].bt;

            totalTAT += processes[idx].tat;
            totalWT += processes[idx].wt;
            completed++;
        }
    }

    cout << "\nID\tAT\tBT\tCT\tTAT\tWT\n";
    for (auto p : processes)
        cout << p.id << "\t" << p.at << "\t" << p.bt << "\t"
             << p.ct << "\t" << p.tat << "\t" << p.wt << "\n";

    cout << "\nAverage TAT = " << totalTAT/n;
    cout << "\nAverage WT = " << totalWT/n << endl;
}
```

Deadlock Detection in Resource Allocation Graph

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
```

```
#define MAX_PROCESSES 10
#define MAX_RESOURCES 10

// Graph structure for cycle detection
int graph[MAX_PROCESSES + MAX_RESOURCES][MAX_PROCESSES + MAX_RESOURCES];
int visited[MAX_PROCESSES + MAX_RESOURCES];
int recStack[MAX_PROCESSES + MAX_RESOURCES];
int totalNodes;

bool dfsCycleDetection(int node) {
    if (!visited[node]) {
        visited[node] = 1;
        recStack[node] = 1;

        for (int i = 0; i < totalNodes; i++) {
            if (graph[node][i]) {
                if (!visited[i] && dfsCycleDetection(i))
                    return true;
                else if (recStack[i])
                    return true;
            }
        }
    }
    recStack[node] = 0;
    return false;
}

bool isDeadlock() {
    for (int i = 0; i < totalNodes; i++) {
        visited[i] = 0;
        recStack[i] = 0;
    }

    for (int i = 0; i < totalNodes; i++) {
        if (dfsCycleDetection(i))
            return true;
    }
    return false;
}

int main() {
    int N, M, E;
    printf("Enter number of processes and resources: ");
    scanf("%d %d", &N, &M);
```

```
    printf("Enter number of edges: ");
    scanf("%d", &E);

    totalNodes = N + M;

    // Initialize graph
    for (int i = 0; i < totalNodes; i++)
        for (int j = 0; j < totalNodes; j++)
            graph[i][j] = 0;

    printf("Enter edges (u v type):\n");
    printf("type 'R' for Request (Process -> Resource)\n");
    printf("type 'A' for Assignment (Resource -> Process)\n");

    for (int i = 0; i < E; i++) {
        int u, v;
        char type[2];
        scanf("%d %d %s", &u, &v, type);

        if (type[0] == 'R' || type[0] == 'r') {
            // Request edge: Process u requests Resource v
            graph[u][N + v] = 1;
        } else if (type[0] == 'A' || type[0] == 'a') {
            // Assignment edge: Resource u assigned to Process v
            graph[N + u][v] = 1;
        }
    }

    if (isDeadlock()) {
        printf("DEADLOCK DETECTED\n");

        // Find which processes are in deadlock
        printf("Processes in deadlock: ");
        for (int i = 0; i < N; i++) {
            if (recStack[i])
                printf("P%d ", i);
        }
        printf("\n");
    } else {
        printf("NO DEADLOCK\n");
    }

    return 0;
}
```

Philosopher Diner with Deadlock

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <stdlib.h>

#define MAX_PHILOSOPHERS 10

typedef struct {
    int id;
    int N;  // Number of philosophers
    sem_t* forks;
    int* eat_count;
} philosopher_t;

void* philosopher_deadlock(void* arg) {
    philosopher_t* phil = (philosopher_t*)arg;
    int id = phil->id;
    int N = phil->N;
    sem_t* forks = phil->forks;

    int left = id;
    int right = (id + 1) % N;

    printf("Philosopher %d is thinking\n", id);
    usleep(rand() % 1000000);

    // Deadlock-prone approach: pick up left fork first, then right
    printf("Philosopher %d is hungry, trying to pick up forks\n", id);

    // Pick up left fork
    sem_wait(&forks[left]);
    printf("Philosopher %d picked up left fork (%d)\n", id, left);
    usleep(100000);  // Small delay to increase chance of deadlock

    // Pick up right fork
    printf("Philosopher %d trying to pick up right fork (%d)\n", id,
right);
    sem_wait(&forks[right]);
    printf("Philosopher %d picked up right fork (%d)\n", id, right);

    // Eating
```

```
    printf("Philosopher %d is EATING\n", id);
    usleep(rand() % 1000000);

    // Put down forks
    sem_post(&forks[left]);
    sem_post(&forks[right]);
    printf("Philosopher %d finished eating and put down forks\n", id);

    return NULL;
}

int main() {
    int N;
    printf("Enter number of philosophers: ");
    scanf("%d", &N);

    if (N > MAX_PHILOSOPHERS) {
        printf("Too many philosophers. Max is %d\n", MAX_PHILOSOPHERS);
        return 1;
    }

    pthread_t philosophers[MAX_PHILOSOPHERS];
    philosopher_t phil_data[MAX_PHILOSOPHERS];
    sem_t forks[MAX_PHILOSOPHERS];

    // Initialize semaphores for forks
    for (int i = 0; i < N; i++) {
        sem_init(&forks[i], 0, 1);
    }

    printf("=== PHASE 1: DEADLOCK DEMONSTRATION ===\n");
    printf("This program will likely deadlock...\n\n");

    // Create philosopher threads
    for (int i = 0; i < N; i++) {
        phil_data[i].id = i;
        phil_data[i].N = N;
        phil_data[i].forks = forks;
        pthread_create(&philosophers[i], NULL, philosopher_deadlock,
&phil_data[i]);
    }

    // Wait for threads (this will likely never complete due to deadlock)
    for (int i = 0; i < N; i++) {
        pthread_join(philosophers[i], NULL);
```

```
    }

    // Cleanup (this won't be reached in deadlock)
    for (int i = 0; i < N; i++) {
        sem_destroy(&forks[i]);
    }

    printf("All philosophers finished (this should not print in
deadlock)\n");

    return 0;
}
```

Without Deadlock

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX_PHILOSOPHERS 10

typedef struct {
    int id;
    int N;
    sem_t* forks;
    sem_t* room;
    bool* eating;
    pthread_mutex_t* print_mutex;
} philosopher_t;

void* philosopher_resolved(void* arg) {
    philosopher_t* phil = (philosopher_t*)arg;
    int id = phil->id;
    int N = phil->N;
    sem_t* forks = phil->forks;
    sem_t* room = phil->room;
    bool* eating = phil->eating;
    pthread_mutex_t* print_mutex = phil->print_mutex;

    int left = id;
    int right = (id + 1) % N;
    int meals = 3;  // Each philosopher eats 3 times
```

```
    for (int meal = 0; meal < meals; meal++) {
        // Thinking
        pthread_mutex_lock(print_mutex);
        printf("Philosopher %d is thinking (meal %d/%d)\n", id, meal + 1,
meals);
        pthread_mutex_unlock(print_mutex);
        usleep(rand() % 500000);

        // Wait to enter the room (limited seating)
        sem_wait(room);

        pthread_mutex_lock(print_mutex);
        printf("Philosopher %d entered the room, waiting for forks\n", id);
        pthread_mutex_unlock(print_mutex);

        // Pick up forks - use ordered approach to prevent deadlock
        if (id % 2 == 0) {
            // Even-numbered philosophers pick up left first
            sem_wait(&forks[left]);
            usleep(100000);  // Small delay to show interleaving

            sem_wait(&forks[right]);
        } else {
            // Odd-numbered philosophers pick up right first
            sem_wait(&forks[right]);
            usleep(100000);

            sem_wait(&forks[left]);
        }

        // Eating
        pthread_mutex_lock(print_mutex);
        eating[id] = true;
        printf("Philosopher %d is EATING (meal %d/%d) - Forks: %d and
%d\n",
               id, meal + 1, meals, left, right);

        // Show current eating status
        printf("Currently eating: ");
        for (int i = 0; i < N; i++) {
            if (eating[i])
                printf("%d ", i);
        }
        printf("\n");
```

```
        pthread_mutex_unlock(print_mutex);

        usleep(rand() % 500000);

        // Put down forks
        sem_post(&forks[left]);
        sem_post(&forks[right]);

        // Leave the room
        sem_post(room);

        pthread_mutex_lock(print_mutex);
        eating[id] = false;
        printf("Philosopher %d finished eating and left the room\n", id);
        pthread_mutex_unlock(print_mutex);
    }

    return NULL;
}

int main() {
    int N;
    printf("Enter number of philosophers: ");
    scanf("%d", &N);

    if (N > MAX_PHILOSOPHERS) {
        printf("Too many philosophers. Max is %d\n", MAX_PHILOSOPHERS);
        return 1;
    }

    pthread_t philosophers[MAX_PHILOSOPHERS];
    philosopher_t phil_data[MAX_PHILOSOPHERS];
    sem_t forks[MAX_PHILOSOPHERS];
    sem_t room;
    bool eating[MAX_PHILOSOPHERS] = {false};
    pthread_mutex_t print_mutex = PTHREAD_MUTEX_INITIALIZER;

    // Initialize semaphores
    for (int i = 0; i < N; i++) {
        sem_init(&forks[i], 0, 1);
    }

    // Room capacity = N-1 (prevents circular wait)
    sem_init(&room, 0, N - 1);
```

```
    printf("\n=== PHASE 2: DEADLOCK PREVENTION ===\n");
    printf("Using limited seating (%d philosophers, %d seats)\n", N, N-1);
    printf("Each philosopher will eat 3 times\n\n");

    // Create philosopher threads
    for (int i = 0; i < N; i++) {
        phil_data[i].id = i;
        phil_data[i].N = N;
        phil_data[i].forks = forks;
        phil_data[i].room = &room;
        phil_data[i].eating = eating;
        phil_data[i].print_mutex = &print_mutex;
        pthread_create(&philosophers[i], NULL, philosopher_resolved,
&phil_data[i]);
    }

    // Wait for all philosophers to finish
    for (int i = 0; i < N; i++) {
        pthread_join(philosophers[i], NULL);
    }

    printf("\n=== SIMULATION TERMINATED ===\n");
    printf("All philosophers have finished eating. No deadlock
occurred!\n");

    // Cleanup
    for (int i = 0; i < N; i++) {
        sem_destroy(&forks[i]);
    }
    sem_destroy(&room);
    pthread_mutex_destroy(&print_mutex);

    return 0;
}
```